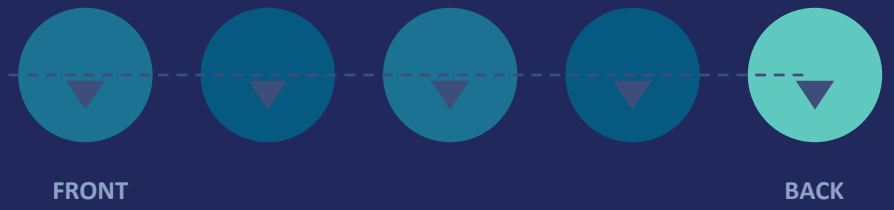


DATA STRUCTURES FOR SERVICE OPERATIONS

Fair Request Processing

Choosing the Queue ADT to Guarantee First-Come, First-Served Order

Technical Presentation | Abstract Data Types in Practice

S Sahithya
192472186

THE PROBLEM

Requests Arrive Continuously — Order Must Stay Fair

A customer service system receives requests from many channels at once — calls, chats, emails, tickets. Whoever designs the system must decide: which request gets handled next?

1

Requests arrive at unpredictable times

The system can't know in advance how many will come or when.

2

Every customer expects equal treatment

No one should be served out of turn without a clear, justified reason.

3

Processing happens one at a time

A single agent or process handles one request before moving to the next.

THE CORE QUESTION

"Which data structure lets us process requests in the exact order they arrived?"

Fairness here means:
First request in → First request served.

FOUNDATIONS

What Is an Abstract Data Type?

An ADT defines what operations a data collection supports and how they behave — independent of how it is implemented in code.

1

Behavior, Not Code

An ADT is a contract: it specifies allowed operations and their rules, not the underlying storage.

2

Implementation-Independent

The same ADT can be built with arrays, linked lists, or other structures underneath.

3

Chosen to Match the Problem

The right ADT is the one whose built-in rules mirror the real-world rule we need to enforce.

Design principle: pick the ADT whose native ordering rule already matches the fairness rule your system needs to enforce.

THE SOLUTION

The Queue ADT

A linear collection that follows one strict rule:

FIFO — First In, First Out

- Elements are added only at the rear (the back of the line).
- Elements are removed only from the front (the head of the line).
- The relative arrival order of every element is always preserved.
- No element can move ahead of one that arrived earlier.

Mapped to customer service:

Enqueue = a new request joins the back of the line
Dequeue = the agent serves the request at the front

HOW IT WORKS

Core Queue Operations

Enqueue(x)

O(1)

Adds a new request x to the rear of the queue.

Dequeue()

O(1)

Removes and returns the request at the front — the one that has waited longest.

Peek() / Front()

O(1)

Looks at the next request to be served, without removing it.

isEmpty()

O(1)

Reports whether any requests are currently waiting.

WHY IT'S FAIR

How FIFO Guarantees Fairness

The queue's structural rule directly enforces the fairness rule the business needs — nothing extra has to be coded.

Requirement	Queue (FIFO) Guarantee
Serve requests in arrival order	Dequeue always returns the element inserted earliest
No request can jump ahead	Insertion happens only at the rear; front position is earned by waiting
Equal treatment for all customers	The ADT applies the same rule to every element — no special-casing
Predictable, explainable order	Order is deterministic: arrival time alone decides service order

COMPARISON

Queue vs. Other Abstract Data Types

Stack

LIFO

Last In, First Out — the newest request would be served first. That rewards recent arrivals and starves early ones.

Priority Queue

BY PRIORITY

Serves by urgency, not arrival time. Useful for SLA tiers, but not for pure arrival-order fairness on its own.

Plain Array / List

NO BUILT-IN ORDER RULE

Any position can be inserted or removed — order must be manually enforced and policed everywhere in the code.

Queue

FIFO

Enforces arrival order intrinsically. The structure itself makes out-of-order service impossible.

BEST FIT

IMPLEMENTATION

Implementing the Queue Efficiently

The ADT's contract stays the same; only the underlying storage changes.

Circular Array

A fixed-size buffer with front/rear indices that wrap around, avoiding wasted space from shifting elements.

Linked List

Each request is a node; the queue keeps head and tail pointers so both ends update in constant time.

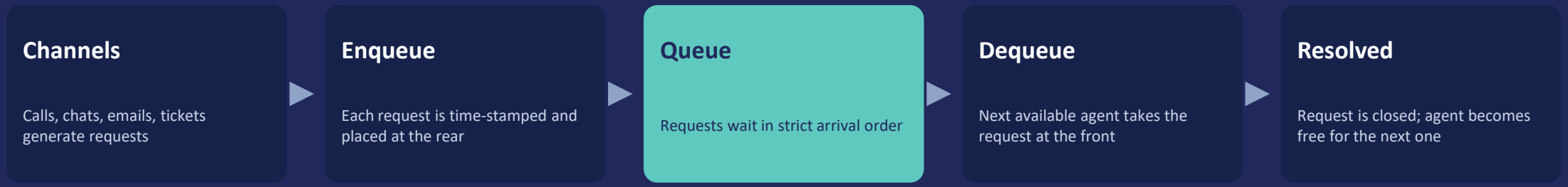
TIME COMPLEXITY SUMMARY

Operation	Array (Circular)	Linked List
Enqueue	O(1)	O(1)
Dequeue	O(1)	O(1)
Peek	O(1)	O(1)

APPLIED

Queue ADT in a Service System

A simplified view of how requests flow through a support desk.



Why this matters operationally:

Customers can trust that waiting longer never disadvantages them. Support teams get a transparent, auditable order that's easy to explain, monitor, and report on — with no risk of silent favoritism.

CONCLUSION

Queue Is the Right ADT for the Job

1. Customer service requires processing requests strictly in arrival order.
2. The Queue ADT's FIFO rule enforces exactly that — by definition, not by extra logic.
3. Enqueue and Dequeue both run in constant time, O(1), so fairness costs nothing extra in performance.
4. The result is a system that is fair, predictable, and easy to explain to customers and auditors alike.

Golden Rule

“The customer who arrives first is served first — always.”

Thank you